

Notes for ECE 50877 - Introduction to Applied Cryptography

Zeke Ulrich

September 17, 2026

Contents

<i>Course Description</i>	1
<i>Basis of Cryptography</i>	2
<i>Computational Security</i>	4
<i>Pseudorandom Number Generators</i>	5
<i>Private Key Encryption</i>	6
<i>Stream Ciphers</i>	8
<i>Block Ciphers</i>	9
<i>Message Authentication Code</i>	10
<i>Authenticated Encryption</i>	11
<i>Hash Functions</i>	12

Course Description

Cryptography is an essential component of securing communication and data in a wide range of applications. This course explores fundamental building blocks in cryptography such as ciphers, hash functions, and digital signatures. We will then demonstrate how these building blocks are used practically to secure communications and systems. We will also learn about the power of advanced cryptography to enable complex functionalities with rigorous security guarantees. We will cover fundamental secure multiparty computation protocols, homomorphic encryption, and blockchains, and review their application in building privacy-preserving machine learning. We will also explore social and ethical issues in developing and deploying cryptographic systems.

Basis of Cryptography

Consider the archetypical case in cryptography. Alice wants to send a message to Bob, and wants the contents to remain secret from Vincent. Alice encrypts her message with a secret key s .

An encryption scheme (Gen, Enc, Dec) is a triple of algorithms where

- Key generation $\text{Gen}(1^n)$ outputs a secret key k of length n (also called the security parameter).
- Encryption algorithm $\text{Enc}_k(m) = c$ outputs ciphertext c .
- Decryption algorithm $\text{Dec}_k(c) = m$ outputs plaintext m .

An encryption algorithm is defined over a message space M and a key space K . Requirements:

- Correctness: $\text{Dec}_k(\text{Enc}_k(m)) = m$.
- Security: definition with respect to adversaries.

The secret key k should be hidden from the adversary. The encryption and decryption algorithms themselves should be public; this is Kerckhoffs's principle. This is because the secrecy keys are easy to hide and replace in case of a breach, not so with algorithms. It is also good to have many eyes on an algorithm to expose security flaws.

The key space must be large enough such that a brute force attack is infeasible. Consider a Caesar cipher; the key space is only 26 long and a message so encrypted is quickly bruteforced.

Shannon's Treatment: messages come from some distribution, let M be a random variable for sampling the messages from the message space M . Distribution M is known to the adversary. This captures a priori information about the messages. The ciphertext $c = \text{Enc}_k(m)$ depends on m chosen according to M . k chosen randomly according to Gen. Enc may also use some randomness. Dec is deterministic. These induce a distribution C over the ciphertexts c . The adversary only observes c . Knowledge about m before observing the output of C is captured by M . Knowledge about m after observing the output of C is captured by $M|C$. An encryption scheme (Gen, Enc, Dec) is *Shannon secure* if for every probability distribution for M , every message $m \in M$, and every ciphertext $c \in C$: $\Pr[M = m|C = c] = \Pr[M = m]$.

An alternative definition of secrecy is *Perfect Secrecy*. An encryption scheme is perfectly secure for every pair of messages m_1 and $m_2 \in M$ and every ciphertext $c \in C$ if $\Pr[\text{Enc}_k(m_1) = c] = \Pr[\text{Enc}_k(m_2) = c]$.

Shannon had fingers in many cryptographic pies. Shannon's Theorem says that if (Gen, Enc, Dec) is a perfectly secret encryption scheme with message space M and key space K , then $|K| \geq |M|$. This means

that for an encryption scheme to be perfectly secret, the keys must be as long as the message. The challenge of modern cryptography is to make something just as good as perfect for practical purposes.

Computational Security

A scheme is (t, ε) -secure if any adversary running for time at most t succeeds in breaking the scheme with probability at most ε . However, this definition makes no guarantees for after t ; perhaps the probability of an attacker succeeding rises sharply after t .

A private-key encryption scheme with key length n and for an adversary running for time t , succeeds in breaking the scheme with probability at most $c \frac{t}{2^n}$ (for some constant c).

A better approach to security is to say a scheme is secure if any *efficient* adversary succeeds in breaking the scheme with at most *negligible* probability.

A function f (from natural numbers to non-negative real numbers) is polynomially bounded (or polynomial) if there is a constant c such that $f(n) < n^c$ for all n .

An algorithm A runs in polynomial time if there is a polynomial p s.t. for all inputs $x \in \{0, 1\}^*$, $A(x)$ terminates within $p(|x|)$ steps.

An algorithm is efficient if it runs in polynomial time.

A function $v(n)$ is negligible if for every c , there exists some N such that for all $n > N$, $v(n) \leq \frac{1}{n^c}$.

A scheme is computationally secure if for every probabilistic polynomial-time (PPT) adversary A carrying out an attack, the probability that A succeeds is negligible.

Pseudorandom Number Generators

Cryptography needs a lot of random to be secure. Computers supply a little bit through core temperature, power consumption, mouse movement, and it would be nice to expand those random bits into many more random bits.

Specifically, given n uniformly random bits $x = x_1|x_2|\dots|x_n$, we want a deterministic polynomial-time algorithm G such that $G(x)$ outputs $l(n)$ bits $y = y_1|y_2|\dots|y_{l(n)}$ with $l(n) > n$ and y looks "as good as" a truly random string. $G : \{0, 1\}^n \rightarrow \{0, 1\}^{l(n)}$ is called a pseudorandom generator (PRG). "As good as" means that the string will pass all statistical tests that a truly random string would pass. You can never generate more information from nothing, so the idea of "as good as" means that no efficient computer can tell $G(x)$ and a truly random string apart. The distributions

$$\left\{x \leftarrow \{0, 1\}^{l(n)} : G(x)\right\} \quad (1)$$

and

$$\left\{r \leftarrow \{0, 1\}^{l(n)} : r\right\} \quad (2)$$

are "computationally indistinguishable".

"Computationally indistinguishable" means that no PPT distinguisher algorithm D can tell them apart.

A deterministic algorithm G is called a pseudorandom generator if:

- G can be computed in polynomial time.
- $|G(x)| > |x|$
- $\{x \leftarrow \{0, 1\}^n : G(x)\} \approx_c \left\{U_{l(n)}\right\}$ where $l(n) = |G(0^n)|$

$\approx_c$ denotes computational indistinguishability. $U_{l(n)}$ denotes uniform distribution over $\{0, 1\}^{l(n)}$. $|G(x) = l(n)|$ is the expansion factor of G .

Private Key Encryption

Recall that for OTPs, n bits of key were required. With PRGs, we can create n bits of "good-enough" random out of fewer than n bits. The secret seed for the PRG takes the place of a key.

Consider an efficient eavesdropping adversary observing the ciphertext. The encryption scheme is secure if the adversary can guess better than random with negligible $v(n)$ probability.

We can define security as having indistinguishable encryptions in the presence of an eavesdropper (EAV-secure) thus: a symmetric encryption scheme (Gen, Enc, Dec) is secure if $\forall m_0, m_1 : \{\text{Enc}_k(m_0)\} \approx_c \{\text{Enc}_k(m_1)\}$ for $k \leftarrow \$ \{0, 1\}^n$.

Consider:

- G is a PRG
- $\text{Gen}(1^n) : k \leftarrow \$ \{0, 1\}^n$
- $\text{Enc}_k(m) = m \oplus G(k)$
- $\text{Dec}_k(c) = c \oplus G(k)$

We wish to prove PRG encryption is EAV-secure. We may prove by reduction. We want to show construction Π is secure. We want to show that Π is secure as long as some underlying problem X is hard. The idea is to show that if you break Π then you also solve X . By contrapositive, then if X is hard then Π is also hard.

Proofs by reduction follow this template:

- We want to show a construction Π secure
- We want to show that Π is secure as long as some underlying problem X is hard
- An algorithm that "breaks" Π can be turned into an algorithm that "breaks" X .
- Therefore, if breaking Π would break X , then Π is secure as long as X is hard. Basically, if breaking Π would require an attacker to break X , then breaking Π is at least as hard as breaking X

We will study these attack models:

- Ciphertext only attack: eavesdropper observes multiple ciphertexts between Alice and Bob, possibly of the same message, and has prior knowledge of the distribution of messages sent (e.g. like letter frequency in the English language). However, ideally an adversary would not be able to tell if the same message is sent. However, if an algorithm is deterministic, then the same message produces the same ciphertext. We will deal with this later

- Known plaintext attack: adversary can see plaintext-ciphertext pairs
- Chosen plaintext attack: adversary picks what messages it can see the encryptions of
- Chosen ciphertext attack: adversary can gather information by obtaining the decryptions of chosen ciphertexts.

Model A is stronger than model B, if a cipher secure against A is also secure against B. It is a fact that CTO < KPA < CPA < CCA.

We want to build a PRG encryption scheme that is CPA-secure. Consider pseudorandom functions, "random-looking" functions.

A random function f from n bits to n bits is a function selected uniformly at random from all 2^{n2^n} functions that map n bits to n bits.

A pseudorandom function F is indistinguishable from a random uniform function such that every polynomial time distinguisher D that receives a description of F outputs 1 with "almost" the same probability as when it receives a description of a random function f .

Let F be a pseudorandom function.

$$\text{Gen}(1^n) : k \xleftarrow{\$} \{0, 1\}^n \quad (3)$$

$$\text{Enc}(k, m) : \text{Pick } r \xleftarrow{\$} \{0, 1\}^n . \text{Output}(r, m \oplus F_k(r)) \quad (4)$$

$$\text{Dec}(k, (r, c)) : \text{Output } c \oplus F_k(r) \quad (5)$$

The above construction is a CPA-secure, fixed-length private-key encryption scheme for messages of length n .

Stream Ciphers

We have seen thus far how to build EAV-secure and CPA-secure encryptions. However, there are some practical issues, such as that they are only defined for fixed-length inputs/outputs.

A stream cipher generates output bits on demand. A stream cipher is a pair of deterministic algorithms (**init**, **next**). **init** takes input seed s and an initialization vector IV and outputs initial state st . **next** takes current state st and outputs a bit y and updated state st' . A stream cipher can be constructed from a pseudorandom function.

Block Ciphers

We use block ciphers to encrypt long (variable input length) messages. We define a block cipher as a pseudorandom permutation (PRP). PRP is a pseudorandom function with the constraint that the entries in any two distinct rows (in a function look-up table) must be different. A PRP is indistinguishable from a random permutation. $2^n!$ random permutations exist for n bits.

Block ciphers have different modes:

- Electronic Code Book (ECB)
- Cipher Block Chaining (CBC)
- Output Feedback (OFB)
- Counter (CTR)

If we use the electronic code book (ECB) mode of operation where we chunk the plaintext and apply the block cipher to each plaintext block, this isn't even EAV-secure since it's deterministic. If a block is repeated in plaintext, it will result in a repeating block in the ciphertext. This could leak a significant amount of information. Don't use the ECB mode!

Cipher block chaining, where the plaintext is XOR'd with the ciphertext of the previous block before encryption, can be shown to be CPA-secure.

Output feedback, where the plaintext is XOR'd with the ciphertext after encryption, is also CPA-secure. In that case the bulk of the computation can be done independently of the message.

In counter mode, follow the steps in Figure ??.

Look up "ECB Penguin" for a visual demonstration of how insecure ECB is.

Not that kind of nonce.

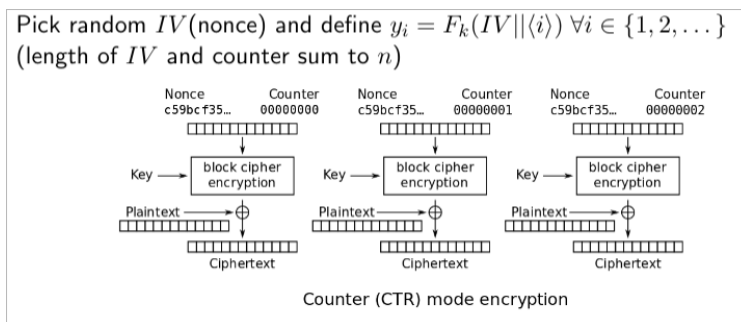


Figure 1: CTR Mode

Message Authentication Code

Encryption ensures confidentiality, but not integrity. A MITM might tamper with the ciphertext.

A *message authentication code* (MAC) consists of three PPT algorithms (**Gen**, **Mac**, **Verify**), such that:

- **Gen**(1^n) outputs a key k
- **Mac** $_k(m)$ takes as input a key k and a message $m \in \{0,1\}^*$ and outputs a tag t .
- **Verify** $_k(m, t)$ takes as input a key k , a message m , and a tag t , and outputs 1 (accept) or 0 (reject)

A MAC allows a recipient to validate that a message was not tampered with and that it was sent by a key holder. We must require

- Correctness: if MAC comes from the authorized source, it should verify.
- Unforgeability: only authorized source can generate the MAC.

An attacker can resend previously authenticated message along with its (valid) tag. This can be prevented with a nonce like a counter or timestamp.

Authenticated Encryption

So far we have considered security and integrity separately. Encryption thus far has not guaranteed integrity, and message authentication codes do not guarantee confidentiality. In fact, in many a scenario, lack of integrity implies lack of confidentiality.

This leads to the need for *authenticated encryption*, which incorporates a checksum. A simple way to do this is tag the message with a checksum and then encrypt both the message and tag (auth-then-enc). You may also send the MAC for the plaintext with the encrypted message (auth-and-enc). It turns out the only secure method is to encrypt and then compute the MAC for the encrypted message. This method, enc-then-auth, is secure. There are still several vulnerabilities:

- Re-ordering attack: swap order of messages
- Replay attack: replay a valid ciphertext previously sent
- Message dropping attack: drop some of the messages sent
- Reflection attack: take a ciphertext from Alice to Bob and send it back to Alice

To defeat these vulnerabilities, keep two counters $\mathbf{ctr}_{A,B}$, $\mathbf{ctr}_{B,C}$ and define a directionality bit b . A sends $c = \text{Enc}_k(b_{A,B}|\mathbf{ctr}_{A,B}|m)$.

Hash Functions

A hash function H has

- deterministic mapping from input to output
- input (long) binary strings
- output short fixed-length string (aka message digest)

A good hash function has as few collisions as possible. In data structures we aim to minimize collisions, but in cryptography we must avoid them. Since the range of a hash function is less than the domain (the output string is shorter than the input string), it's impossible to completely avoid collisions, but finding x_1, x_2 s.t. $H(x_1) = H(x_2)$ should be computationally infeasible.

We consider keyed hash functions with the difference that keys are public.

$$H^s(x) = H(s, x) \quad (6)$$

A hash function with output length $l(n)$ is a pair of PPT algorithms (Gen, H) s.t. Gen takes as input a security parameter 1^n and outputs a key s and H is a deterministic algorithm that takes as input a key s and a string $x \in \{0, 1\}^*$ and outputs a string $H^s(x) \in \{0, 1\}^{l(n)}$.

Collision resistance states that no PPT adversary can find a collision except with negligible probability. In practice, we use unkeyed hash functions with a fixed output length. Such hash functions are collision resistant for all practical purposes (colliding pairs are unknown and difficult to find).

For some applications it might make sense to use a weaker notion than collision resistance.

- Preimage resistance: Given s and $y = H^s(x)$ for a uniform x , it is infeasible for a PPT adversary to find a value x' (equal to x or not) such that $H^s(x') = y$. In other words, H^s is a one-way function.
- Second-preimage resistance: Given s and a uniform x it is infeasible for a PPT adversary to find $x' \neq x$ such that $H^s(x') = H^s(x)$.

A collision resistant hash function is also second-preimage resistant.

A second-preimage resistant hash function is also preimage resistant (under certain conditions).

Evaluating H with digest length l on $q \approx 1.2 \times 2^{\frac{l}{2}}$ yields a collision with probability $\frac{1}{2}$. For n -bit of effective security need digest length of at least $2n$.